

Palantír Development Guide

This is the technical companion to `README.md`. The README tells you how to run and publish the app. This document explains why it's built the way it is, how the pieces fit together, and what has already bitten us once, so it doesn't bite twice.

1. What this is, briefly

A personal work console, built as a real installable Windows desktop app (Electron), not a website. It replaced Google Tasks / MS To Do because those tools flatten everything into an identical flat checklist, while this one mirrors how work actually has shape: projects, status, priority, dependencies between sub-items, recurrence, and history.

Single user, single machine by default, no account, no server. All data is one JSON file on disk.

2. How we got here (project history)

Worth knowing because it explains some scars in the current setup:

1. **Started as a scoping exercise**, not a build. The goal was deliberately clarified through structured back-and-forth before any code was written: what's actually wrong with Google Tasks/MS To Do, what "using GitHub" should mean, what the UI should look like, before touching a keyboard.
2. **Explicitly kept separate from "Anchor."** Anchor is a different, unrelated personal project (an ADHD/executive-function-focused Android home-screen widget, native, zero-decision capture). Early on we considered merging them into one system and deliberately rejected it, the interaction models are fundamentally incompatible (a widget requires OS-level native integration; this app is a general desktop console). If a future session suggests merging them again, that's been considered and turned down once already, for good reason.
3. **First build was a website**, React/Vite, intended to deploy on GitHub Pages with data stored via the GitHub API (commits to a repo as the "database"). This actually got built, including a working GitHub API client with UTF-8-safe base64 handling for Hebrew text.
4. **Pivoted entirely to desktop.** The GitHub Pages + Actions + repo-visibility + Pages-source setup chain turned out to be a lot of fragile, unfamiliar steps for what should have been a simple personal tool. Rather than keep debugging deployment, we rebuilt as an Electron app with local file storage. This was the right call, the amount of setup friction dropped enormously, but it means any stale references to "GitHub Pages" or "the web version" elsewhere are historical, not current.

5. **Delivered since then in incremental batches**, each fully built, tested, and shipped before starting the next:
- **Quick Wins** (UI polish): keyboard shortcuts, command palette, inline rename, RTL-aware text fields.
 - **Functionality**: Today-as-default view, free-text search, bulk actions, per-item history log.
 - **Desktop-only capabilities**: global quick-capture hotkey, system tray + launch-at-login, real file attachments, native notifications for recurring tasks.
 - **Foundational**: daily automatic backups, undo-for-deletes, auto-update via GitHub Releases.

3. Architecture overview

Three layers, standard Electron shape:

- `electron/main.cjs` — the main (Node) process. Owns the filesystem, the tray, the global hotkey, all windows, and all IPC handlers. This is the only place with real Node/OS access.
- `electron/preload.cjs` — the security bridge. Uses `contextBridge` to expose a narrow `window.palantir` API to the renderer. `contextIsolation: true` and `nodeIntegration: false` are set deliberately; the renderer never gets raw Node access.
- `src/` — the React 19 + TypeScript + Vite + Tailwind renderer. This is the actual UI. It talks to the outside world only through `window.palantir.*` methods (see §5).

Two windows, one codebase

The same built `dist/index.html` is loaded by two different `BrowserWindow`s:

- The **main window** — the full app (`App.tsx`).
- The **quick-capture popup** — a small frameless always-on-top window, loaded with `?mode=quickcapture` in the URL. `main.tsx` reads that query param and renders `QuickCapture.tsx` instead of the full `App` when it's present. Triggered by the global hotkey (`Ctrl/Cmd+Shift+Space`) or the tray menu.

Keeping two windows in sync

Quick capture writes directly to the data file from the **main process** (not through the main window's React state, since the main window might not even be open). After writing, main process sends a `palantir:data-changed` event to the main window if it exists;

`DataContext` listens for this and reloads from disk. This is the mechanism that makes "type a task while the app is minimized" actually show up correctly when you reopen it.

Why the app doesn't quit on window close

`mainWindow.on('close', ...)` intercepts the close event and hides the window instead of quitting, so the tray icon, global hotkey, and quick capture keep working even when you've "closed" the app. Real quit only happens via the tray menu's "Quit," which sets an `isQuitting` flag first. **This trips people up during development too:** if you leave an old instance running in the tray and then run `npm run electron:dev` again, you now have two instances. Quit the tray one first.

Data model and migrations

Everything lives in `src/types.ts`. Current shape (abbreviated):

```
1 PalantirData
2   | projects: Project[]           { id, name, createdAt }
3   | workItems: WorkItem[]
4     { id, title, status, priority, projectId, notes,
5       links: Link[], subItems: SubItem[], attachments: Attachment[],
6       recurrence: Recurrence | null, workday: boolean,
7       history: HistoryEntry[],
8       createdAt, startedAt, completedAt, updatedAt }
9
```

- `SubItem` has its own `priority` (L/M/H/U) but no independent status, just a checkbox.
- `history` is an append-only log of status transitions, separate from `startedAt` / `completedAt` (which only ever hold the *most recent* transition of each kind). History survives a task bouncing back and forth between statuses; the two timestamp fields don't.
- `attachments` are copies of real files, stored outside the JSON, referenced by id (see §7).

The migration system — read this before adding any field

`migrateData()` in `types.ts` runs on every app load and on every backup import. Its job: take data that might have been saved by an *older* version of the app (missing fields that didn't exist yet) and backfill sensible defaults, without touching data that's already current.

The rule going forward: any time a new field is added to `WorkItem` or `SubItem`, `migrateData()` must be updated to backfill it, or the app will throw at runtime on anyone's existing data file. TypeScript's types don't protect you here, they're compile-time only; a real JSON file from before the field existed will genuinely lack it at runtime, and code like `PRIORITY_META[item.priority].label` will crash on `undefined`.

This has been tested for real each time, not just assumed: sub-item priority, history, and attachments were each verified by hand-writing a fake "old" data file missing the new field, running the app against it, and confirming it upgrades cleanly and writes back the corrected shape.

IPC surface (the full `window.palantir` API)

Method	Direction	Purpose
<code>loadData()</code> / <code>saveData(data)</code>	renderer → main	Read/write the main data file
<code>getDataPath()</code>	renderer → main	Shown in Settings
<code>exportData(data)</code> / <code>importData()</code>	renderer → main	Manual backup, via native file dialogs
<code>onDataChanged(cb)</code>	main → renderer	Fired after quick capture writes, tells the main window to reload
<code>quickCapture(title)</code> / <code>closeQuickCapture()</code>	renderer → main	Used only by the quick-capture popup
<code>getLaunchAtLogin()</code> / <code>setLaunchAtLogin(bool)</code>	both	OS login-item setting
<code>addAttachment(id)</code> / <code>openAttachment(..)</code> / <code>removeAttachment(...)</code> / <code>deleteAttachmentsFolder(id)</code>	renderer → main	File attachment lifecycle
<code>revealDataFile()</code> / <code>revealBackupsFolder()</code>	renderer → main	Opens File Explorer via <code>shell</code>
<code>getAppVersion()</code>	renderer → main	Settings display

<code>checkForUpdates())/ quitAndInstall()</code>	renderer → main	Manual update trigger and restart-to-install
<code>onUpdateStatus(cb)</code>	main → renderer	Update state machine: checking / downloading / ready / up-to-date / error / dev-mode

Every one of these is defined in three places that must stay in sync: `electron/main.cjs` (the real `ipcMain.handle()`), `electron/preload.cjs` (the `contextBridge.exposeInMainWorld` wrapper), and `src/electron.d.ts` (the TypeScript ambient type). Adding a new IPC method means touching all three.

Key design decisions (and the reasoning, so they don't get silently re-litigated)

- **Inbox** is the mandatory catch-all project; every task belongs to exactly one project, and Inbox exists so personal/unordered items always have a home.
- **Filters are OR within a category, AND across categories.** Priority: Urgent-or-High, and Project: Amit, and Status: To do, together.
- **Sort has a per-field default direction**, not a universal one: Priority defaults to Urgent→Low, Status defaults to workflow order (To do→Complete), Project defaults A→Z. Clicking an already-active sort field flips its direction; clicking a different field resets to *that* field's own default. This was worked out explicitly with the user rather than assumed.
- **"Workday" (the Today tag) resets on first app-open of a new day**, not at literal midnight. There's no background process in a plain desktop app to fire something at exactly 00:00, so this is an honest, documented approximation: the check runs in `DataContext` on load, compares today's date against a `localStorage` marker, and clears the flags if they don't match.
- **The app opens on the Today view by default**, not "everything." Deliberate, matches how someone actually starts a work session.
- **Search matches title and notes only**, not sub-items or links. A scoping choice, not an oversight.
- **Bulk delete asks for confirmation (native `confirm()`); single delete does not**, it relies on the undo toast instead. Two different safety mechanisms for two different blast radii, both intentional.

- **Undo is single-slot:** only the most recent delete (single or bulk) can be undone, for 8 seconds. A second delete while one is still pending immediately finalizes the first (its attachments get cleaned up for real) before staging the new one. This was a real bug caught and fixed during building, an earlier version would have silently orphaned the first deletion's attachment files.
- **Recurrence has no time-of-day.** Only daily or specific-weekday recurrence exists; there is no "remind me at 9am" concept, because that needs a real scheduling data model that hasn't been designed. This has been explicitly deferred **twice** (once when discussing functionality, once when discussing notifications) — if it comes up again, it's not an oversight, it's waiting on a real design decision about what "scheduled" should mean here.
- **Quick capture bypasses the renderer entirely for the write.** It's handled in the main process directly against the file, specifically so it works even when the main window doesn't exist yet.

Where things live on disk

All under Electron's `app.getPath('userData')` (on Windows, roughly `%APPDATA%\palantir\`):

- `palantir-data.json` — the one real data file.
- `backups/palantir-backup-YYYY-MM-DD.json` — one automatic snapshot per calendar day, last 14 kept, older ones pruned automatically.
- `attachments/<workItemId>/<attachmentId>-<originalFileName>` — real copies of attached files, not shortcuts/links. Deleting a task (after its undo window closes) also deletes its attachments folder.

What to avoid: concrete pitfalls already hit

- `EPERM: operation not permitted, rename ...` **during** `npm run dist / npm run release`. Almost always means the app is still running (check the tray, not just open windows) or a Windows/OneDrive/antivirus file lock. Fully quit via the tray menu and delete the half-built `release/` folder before retrying.
- `GH_TOKEN` **does not persist across terminal windows or reboots.** It has to be `set / $env:` in the *same* window you then run `npm run release` in, every time. Confirm it actually took with `echo %GH_TOKEN% / echo $env:GH_TOKEN` before running the release command, don't assume.
- `npm run dist` **is deliberately configured with** `--publish never`. A plain local build should never try to talk to GitHub. Only `npm run release ( --publish always )` does. If

a future change to `package.json`'s scripts removes that flag, local builds will start failing with the same "GitHub token not set" error for no good reason.

- **Never commit a token to any file in the repo.** Tokens are entered into the terminal per-session only.
- **The app is unsigned.** A code-signing certificate costs money and requires identity verification, not worth it for a personal tool. Expect a one-time Windows SmartScreen "Unknown Publisher" warning on fresh installs. Auto-updates don't re-trigger this, since they're applied by the already-trusted running app.
- **Vite's base must stay `'./'` (relative), not `'/'`.** Electron loads the built `index.html` via `file://`, not `http://`; an absolute base path silently breaks every asset reference. This was a real bug during the GitHub-Pages-to-desktop pivot.
- **The Gadpal/Palantir GitHub repo's actual file contents are stale**, left over from the abandoned web-app iteration. It's still the correct repo for publishing Releases (release publishing doesn't care what's in the repo), but don't expect to find current source there unless someone deliberately pushes it.
- **Fine-grained GitHub tokens need the "Contents" permission specifically**, scoped to the one repo. That single permission covers both the old GitHub-Pages-data-storage use case (now removed) and Releases publishing (current use case).

Explicitly out of scope right now

So nobody re-decides these from scratch without knowing they were already considered:

- **Time-of-day scheduling / reminders** — needs a real scheduling model first (see §6).
- **Merging with Anchor** — considered and rejected once already (see §2).
- **Dark theme** — explicitly excluded when the UI-polish batch was scoped; everything else in that batch was approved.
- **Code signing** — not worth the cost/complexity for personal use (see §8).
- **Cross-device sync** — data is single-machine by design; Export/Import backup is the manual workaround.

Stack reference

Electron 43, React 19, TypeScript, Vite 8, Tailwind CSS 3, `@dnd-kit` (drag-to-reorder sub-items), `lucide-react` (icons), `@fontsource/*` (self-hosted Inter/Spectral/JetBrains Mono, no CDN dependency so the app works offline), `electron-builder` (packaging + NSIS installer), `electron-updater` (auto-update against GitHub Releases).